

Contracts

Smart Contracts Specification

This document provides a comprehensive reference for every deployed contract in the UnifyVault V2 protocol.

Contract Inventory

- **Contract Name:** License | Inheritance | Key Responsibility
- **`ProtocolDirectory`:** MIT | `AccessControl`, `IProtocolDirectory` | Dynamic address registry for protocol modules
- **`UnifyVaultControllerUpgradeable`:** MIT | `Initializable`, `AccessControlUpgradeable`, `ReentrancyGuard`, `PausableUpgradeable`, `UUPSUpgradeable` | Core UUPS execution orchestrator for deposits, redemptions, and NAV
- **`CustodyVault`:** MIT | `AccessControl`, `ReentrancyGuard`, `Pausable`, `IVault` | Secure collateral custody with donation-attack immunity
- **`Treasury`:** MIT | `AccessControl`, `ReentrancyGuard`, `Pausable`, `ITreasury` | Protocol fee vault and treasury asset balance tracking
- **`FeeManager`:** MIT | `AccessControl`, `IFeeManager` | Deposit/redemption fee computation and fee routing to Treasury
- **`LiquidityManager`:** MIT | `AccessControl`, `ReentrancyGuard`, `ILiquidityManager` | Liquidity reserve monitoring and vault refill/sweep execution
- **`OracleManager`:** MIT | `AccessControl`, `IOracle` | Multi-feed price aggregator with staleness and fallback routing
- **`ChainlinkOracleProvider`:** MIT | `AccessControl`, `IOracleProvider` | Chainlink `AggregatorV3Interface` price feed adapter
- **`MockOracleProvider`:** MIT | `AccessControl`, `IOracleProvider` | Controlled oracle provider for test environments and emergency pricing
- **`StrategyManager`:** MIT | `AccessControl`, `IStrategyManager` | Target asset allocation (BPS) management
- **`PortfolioManager`:** MIT | `AccessControl`, `ReentrancyGuard`, `IPortfolioManager` | Portfolio NAV computation and automated asset rebalancing
- **`SwapAdapter`:** MIT | `AccessControl`, `ISwapAdapter` | DEX router adapter enforcing slippage protection
- **`UVBEV2`:** MIT | `ERC20`, `ERC20Permit`, `AccessControl`, `Pausable`, `IToken` | Index share token with locked pre-transfer cost basis hooks
- **`CostBasisManagerV2`:** MIT | `AccessControl`, `ICostBasisManagerV2` | Realized/unrealized P&L, entry price, and P2P/Staking transfer filter
- **`PerformanceManager`:** MIT | `AccessControl`, `IPerformanceManager` | Benchmark tracking, high-water marks, and time-weighted returns
- **`P2PEscrowV2`:** MIT | `AccessControl`, `ReentrancyGuard`, `Pausable`, `IP2PEscrow` | Non-custodial crypto-fiat escrow with cryptographic proof verification
- **`Marketplace`:** MIT | `AccessControl`, `ReentrancyGuard`, `Pausable` | Non-custodial limit orderbook engine with linked escrow creation
- **`GasTreasury`:** MIT | `AccessControl`, `ReentrancyGuard` | ERC-4337 paymaster deposit holding and sponsorship pool
- **`UnifyVaultTimelock`:** MIT | `TimelockController` | 48-hour delay timelock controller for governance execution

1. ProtocolDirectory

Purpose

Serves as the central service locator for the UnifyVault protocol, allowing contracts to discover sibling modules dynamically.

Roles

- ``DEFAULT_ADMIN_ROLE``: Role management.
- ``GOVERNANCE_ROLE``: Can call ``registerAddress``, ``updateAddress``, ``removeAddress``, and ``freeze``.

Key Public Methods

- ``registerAddress(bytes32 id, address target)``: Registers a new module.
- ``updateAddress(bytes32 id, address target)``: Updates an existing module address.
- ``removeAddress(bytes32 id)``: Deletes a module entry.
- ``freeze()``: Permanently disables registry modifications.
- ``getAddress(bytes32 name)`` returns `(address)``: Fetches target address; reverts if non-existent.
- ``exists(bytes32 name)`` returns `(bool)``: Returns whether module ID is registered.
- ``isFrozen()`` returns `(bool)``: Returns true if registry is frozen.

Events

- ``AddressRegistered(bytes32 indexed id, address indexed target, address indexed actor)``
 - ``AddressUpdated(bytes32 indexed id, address indexed oldTarget, address indexed newTarget, address indexed actor)``
 - ``AddressRemoved(bytes32 indexed id, address indexed oldTarget, address indexed actor)``
 - ``RegistryFrozen(address indexed actor)``
-

2. UnifyVaultController

Purpose

The primary interface for users to deposit collateral, receive shares, redeem shares for collateral, and trigger rebalancing workflows.

Roles

- ``DEFAULT_ADMIN_ROLE``: Role management.
- ``GOVERNANCE_ROLE``: Manages deposit limits, rate limits, slippage parameters, and monitoring thresholds.
- ``GUARDIAN_ROLE``: Can trigger ``pause()`` and ``unpause()``.
- ``BOT_ROLE``: Can trigger automated keeper functions.

Key Public Methods

- ``deposit(address asset, uint256 amount, uint256 minSharesOut, address receiver)`` returns `(DepositQuote memory)``: Deposits collateral asset and mints shares.
 - ``redeem(address asset, uint256 shares, uint256 minAssetsOut, address receiver, uint256 deadline)`` returns `(uint256)``: Burns shares and returns net collateral.
 - ``rebalancePortfolio(address[] targetAssets, uint256[] targetWeights)``: Rebalances portfolio assets via ``PortfolioManager``.
 - ``setMaxDeposit(uint256 maxDeposit)``: Updates max deposit per tx.
 - ``setDepositLimits(uint256 maxPerTx, uint256 dailyCap)``: Updates deposit caps.
 - ``setRedeemLimits(uint256 maxPerTx, uint256 dailyCap)``: Updates redemption caps.
 - ``setSwapSlippageBps(uint256 slippageBps)``: Sets maximum swap slippage tolerance.
 - ``pause()`` / ``unpause()``: Emergency circuit breaker.
-

3. CustodyVault

Purpose

Holds underlying collateral assets securely, enforcing accounting isolation against raw transfers (donation-attack immunity).

Roles

- ``DEFAULT_ADMIN_ROLE``: Role management.
- ``GOVERNANCE_ROLE``: Registers/enables/disables supported assets.
- ``CONTROLLER_ROLE``: Authorized to call ``depositAsset`` and ``withdrawAsset``.

Key Public Methods

- ``registerAsset(address asset, uint8 decimals)``: Whitelists asset for custody.
 - ``depositAsset(address asset, uint256 amount)``: Records deposit and transfers tokens from user.
 - ``withdrawAsset(address asset, uint256 amount, address recipient)``: Transports assets to recipient.
 - ``totalAssets(address asset)`` returns (uint256): Returns tracked internal balance.
 - ``surplusAssets(address asset)`` returns (uint256): Returns untracked balance (donation buffer).
-

4. Treasury & FeeManager

Purpose

``Treasury`` securely stores collected protocol fees and reserves. ``FeeManager`` calculates fees on deposits and redemptions and routes them to Treasury.

Key Public Methods

- ``FeeManager.calculateDepositFee(uint256 amount)`` returns (uint256): Computes deposit fee.
 - ``FeeManager.calculateRedemptionFee(uint256 amount)`` returns (uint256): Computes redemption fee.
 - ``Treasury.collectFee(address asset, uint256 amount)``: Stores collected fees.
 - ``Treasury.withdraw(address asset, uint256 amount, address recipient)``: Withdraws treasury earnings (governance-only).
-

5. LiquidityManager

Purpose

Monitors vault asset reserves against minimum/maximum thresholds and executes rebalance sweeps or refills.

6. OracleManager & ChainlinkOracleProvider

Purpose

Aggregates pricing from Chainlink price feeds (``ChainlinkOracleProvider``) and fallback sources, enforcing heartbeat limits and price freshness bounds.

Key Public Methods

- ``configureAsset(bytes32 assetId, address primaryProvider, address fallbackProvider, uint256 heartbeat, bool enabled)``: Sets oracle feed parameters.
 - ``getAssetPrice(address asset)`` returns (uint256): Returns normalized asset price in 18 decimals.
 - ``isPriceFresh(address asset)`` returns (bool): Returns true if price timestamp is within heartbeat limits.
-

7. StrategyManager & PortfolioManager

Purpose

`StrategyManager` maintains target portfolio allocation weights (in BPS). `PortfolioManager` evaluates current NAV, computes allocation drift, calculates UVBE token price, and executes rebalance swaps through `SwapAdapter`.

Key Public Methods

- `calculateNAV()` returns (uint256): Calculates total portfolio USD valuation in 18 decimals.
- `calculateUVPrice()` returns (uint256 totalValueUSD, uint256 uvPriceUSD): Computes share valuation (NAV / totalSupply).
- `rebalance(address[] targetAssets, uint256[] targetWeights)`: Performs asset swaps to restore target portfolio weighting.

8. SwapAdapter

Purpose

Wraps DEX routers (e.g. Uniswap V3) to perform token swaps with guaranteed minimum output bounds computed using `OracleManager` prices and `swapSlippageBps`.

9. UVBEV2 (UVBEToken)

Purpose

ERC20 share token representing ownership in the vault, enhanced with `ERC20Permit` and a locked pre-transfer cost basis hook.

Architecture

- Overrides `\_update(from, to, value)`:
 1. Reads `senderBalanceBefore`.
 2. Calls `costBasisManager.onTokenTransfer(from, to, value, senderBalanceBefore)` BEFORE updating balances.
 3. Executes `super.\_update(from, to, value)`.
- Minting and burning are strictly restricted to `CONTROLLER\_ROLE`.

10. CostBasisManagerV2

Purpose

Maintains on-chain investor cost basis, entry prices, realized P&L, and unrealized returns.

Key Features

- **Transfer Hook** (`onTokenTransfer`)**: Shifts proportional cost basis on peer-to-peer transfers, maintaining exact basis conservation across the protocol.
- **P2P Escrow Exclusion** (`\_isEscrow`)**: When tokens transfer to or from registered escrow contracts, the hook returns immediately, ensuring P2P fiat trades do not contaminate portfolio investment basis.
- **Realized P&L** (`recordRedeem`)**: Calculates realized gain/loss upon share redemption.
- **View Methods****:
 - `costBasis(address account)` returns (uint256)
 - `averageEntryPrice(address account)` returns (uint256)
 - `realizedPnL(address account)` returns (int256)
 - `unrealizedPnL(address account)` returns (int256)

11. PerformanceManager

Purpose

Computes portfolio benchmark returns, high-water marks, and historical performance tracking.

12. P2PEscrowV2 & Marketplace

Purpose

Provides a non-custodial limit orderbook and escrow clearinghouse for peer-to-peer trading of UVBE and crypto assets against fiat (INR/USD).

Key Features

- **State Machine**: ``CREATED`` & ``FUNDED`` & ``PAYMENT_SUBMITTED`` & ``RELEASED`` / ``REFUNDED`` / ``DISPUTED``.
 - **Replay & Fraud Protection**: Unique ``_usedPaymentReferences`` (UTR) and ``_usedEvidenceHashes`` (receipt cryptographic hashes) prevent double-settlement.
 - **Fee Routing**: Fee (up to 500 BPS max) deducted from released collateral and routed to ``Treasury``.
 - **Arbitration**: Role-based dispute resolution via ``ARBITRATOR_ROLE`` or ``GOVERNANCE_ROLE``.
 - **Zero Vault Exposure**: Pure settlement layer operating strictly on circulating tokens; does not touch vault collateral, supply, or NAV.
-

13. UnifyVaultTimelock

Purpose

Inherits OpenZeppelin ``TimelockController`` to enforce a 48-hour delay on governance proposals.

Configuration

- Minimum delay: ``48 hours`` (``172800`` seconds).
 - Proposer: Configured Gnosis Safe address (``0x1111111111111111111111111111111111111111``).
 - Executor: Open execution (``address(0)``).
-

14. Canonical Deployed Addresses (Base Sepolia - Chain ID 84532)

- **Contract Module**: Canonical Address
- **ProtocolDirectory**: ``0xD2715141a0F5998B707BaA963990bFC2E94cF145``
- **Treasury**: ``0x66182F56BD5E523c655f6890290aB519f528e83f``
- **FeeManager**: ``0x0721465B01b586B7AAdF957A4a884acE46CfbEc9``
- **CustodyVault**: ``0x27B5C6DEA90678B78856b0B10DBA37A789fDe97e``
- **OracleManager**: ``0x5B6067982C6ccE2DC760EB4731c1b40136776D4A``
- **ChainlinkOracleProvider**: ``0x4F7f99653d9d7aCD462429ffFc0C4B6C8Cf4354a``
- **LiquidityManager**: ``0xa938aaCeA64bE8f41c90960aFF232dA4Df7Fc329``
- **UVBEV2 (UVBEToken)**: ``0xA3Db7c3DeE9A50D966A06e19b5DF4FCDee615BdE``
- **UnifyVaultController**: ``0x07f3D3432B64DBF67c5b061AF2bC8Aef70221Cea``
- **StrategyManager**: ``0x14058459198a2CfFc8cE89C364334a80Da82D6a3``
- **PortfolioManager**: ``0x1C65B1667c8cC03138b8e57cDd40b0Bf28a4cDc4``
- **SwapAdapter**: ``0xCb1a434c5ebe2F2F8672Ca507Ee819C6888ae634``
- **CostBasisManagerV2**: ``0xF71706A2Fd8692e3C739855B2A33C0E679b4c382``
- **PerformanceManager**: ``0x133fD024EA635694A223e66B936c2afAB4F2DB78``
- **P2PEscrowV2**: ``0xbAc9C1b440adf74688abBD5be950ABd2766E5B7b``
- **Marketplace**: ``0xe908377f96F313a6b7771570ff6Fb414D38F451A``
- **TimelockController**: ``0x9094145Cd2AEA2f309eDf14237444a07edF98d02``